



Hacer agregaciones desde Python

El uso de **agregaciones** en MongoDB permite realizar análisis complejos sobre los datos, como **transformaciones, filtrado o agrupaciones**. A través de PyMongo, es posible implementar estos *pipelines* directamente desde Python, lo que brinda flexibilidad y poder analítico en el manejo de bases de datos NoSQL. En esta lectura, aprenderás cómo estructurar y ejecutar estos procesos utilizando listas de diccionarios, lo cual facilita la interacción entre Python y MongoDB.

De manera similar al método para acceder a los resultados de consultas, PyMongo provee métodos para recuperar los resultados de agregaciones o *pipelines* en MongoDB.

Recuerda que las agregaciones consisten en un **arreglo de documentos válidos** con la siguiente sintaxis:

```
[etapa1, etapa2, etapa3, ... , etapaN]
```

En Python, el *pipeline* estaría constituido por una **lista de diccionarios**.

Por ejemplo, si retomamos el caso de la lección anterior, en el que queríamos obtener un listado de los ganadores del premio que están vivos y los que han fallecido, la agregación que utilizamos fue la siguiente:

```
[{
    "$addFields":
    {
        "live": {
            "$cond": {
                "if": {
                    "$ifNull": ["$death", false]
                },
                "then": "dead",
                "else": "live"
            }
        }
    }
},
{
    "$group":
    {
        "_id": "$live",
        "total_laureates": {
            "$sum": 1
        }
    }
}]
```

En Python, si asignamos el *pipeline* a una variable, podemos llamar al comando `aggregate` de la colección y convertir el resultado en una lista de forma directa:

```
pipeline = [  
    {  
        '$addField': {  
            'live': {  
                '$cond': {  
                    'if': {  
                        '$ifNull': [  
                            '$death', False  
                        ]  
                    },  
                    'then': 'dead',  
                    'else': 'live'  
                }  
            }  
        }  
    }, {  
        '$group': {  
            '_id': '$live',  
            'total_laureates': {  
                '$sum': 1  
            }  
        }  
    }  
]
```

```
dead_live = list(laureates.aggregate(pipeline))
```

```

✓ [16] 1 pipeline = [
      2     {
      3         '$addFields': {
      4             'live': {
      5                 '$cond': {
      6                     'if': {
      7                         '$ifNull': [
      8                             '$death', False
      9                         ]
     10                     },
     11                     'then': 'dead',
     12                     'else': 'live'
     13                 }
     14             }
     15         }, {
     16             '$group': {
     17                 '_id': '$live',
     18                 'total_laureates': {
     19                     '$sum': 1
     20                 }
     21             }
     22         }
     23     }
     24 ]
     25
     26 dead_live = list(laureates.aggregate(pipeline))
     27
     28 dead_live

```

```

[{'_id': 'dead', 'total_laureates': 626},
 {'_id': 'live', 'total_laureates': 317}]

```

¿Puedes determinar el resultado en Python de la ejecución de la siguiente agregación sobre la colección `awards`?

```
pipeline_awards = [{
    "$match":
    {
        "laureates": {
            "$exists": 1
        }
    }
},
{
    "$group":
    {
        "_id": "$awardYear",
        "total_laureates":
        {
            "$sum": {
                "$size": "$laureates"
            }
        }
    }
},
{
    "$sort":
    {
        "total_laureates": -1
    }
}]
```

```
tol = list(nobel.awards.aggregate(pipeline_awards))
```

```

✓ [19] 1 pipeline_awards = [{
2     "$match":
3     {
4         "laureates": {
5             "$exists": 1
6         }
7     }
8 },
9 {
10    "$group":
11    {
12        "_id": "$awardYear",
13        "total_laureates":
14        {
15            "$sum": {
16                "$size": "$laureates"
17            }
18        }
19    }
20 },
21 {
22     "$sort":
23     {
24         "total_laureates": -1
25     }
26 }]
27
28 tol = list(nobel.awards.aggregate(pipeline_awards))
29
30 tol

```

```

{'_id': '1937', 'total_laureates': 21},
{'_id': '1929', 'total_laureates': 21},
{'_id': '1927', 'total_laureates': 21},
{'_id': '1952', 'total_laureates': 21},
{'_id': '1936', 'total_laureates': 21},

```

✓ 0 s se ejecutó 23:52

Por último, recuerda que los resultados convertidos a lista pueden ser manipulados como cualquier otra lista. Por ejemplo, podríamos convertir el resultado anterior en un *dataframe* de la biblioteca Pandas con la siguiente sintaxis:

```
import pandas as pd
```

```
total_laureates = pd.DataFrame(tol)
```

```
[20] 1 import pandas as pd
      2
      3 total_laureates = pd.DataFrame(tol)
      4
      5 total_laureates
```

	_id	total_laureates
0	2001	45
1	2019	42
2	2013	39
3	2011	39
4	2009	39
...	...	...
111	1917	12
112	1924	9
113	1914	9
114	1918	6
115	1916	3

116 rows x 2 columns

✓ 0 s se ejecutó 23:53

Como puedes observar, las **agregaciones en MongoDB**, ejecutadas desde Python con PyMongo, permiten estructurar consultas avanzadas de forma clara y eficiente. El uso de listas de diccionarios como *pipelines* y su integración con estructuras como *dataframes* hacen de este enfoque una herramienta poderosa para el análisis de datos.

Para terminar...

Acceder a una base de datos mediante un lenguaje de programación es un fundamento clave en muchos de los procesos actuales de manipulación y análisis de datos, los cuales se utilizan para apoyar la toma de decisiones. A partir de esto, reflexiona:

- ¿Puedes identificar procesos de este tipo en una empresa?
- ¿Qué condiciones previas consideras necesarias para implementar procesos de análisis de datos?

Continúa con el siguiente recurso, donde pondrás a prueba los conocimientos adquiridos en esta lección.

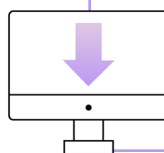
Bibliografía

Bionichive. (s.f.). *BSON Specification* [Especificación BSON]. Bionichive. Recuperado el 11 de junio de 2025 de: <https://www.bionichive.com/>

MongoDB Inc. (s.f.). *Documentación de MongoDB*. MongoDB. Recuperado el 11 de junio de 2025 de: <https://docs.mongodb.com/>

MongoDB Inc. (s.f.). *PyMongo 4.13.0 Documentation*. MongoDB. Recuperado el 11 de junio de 2025 de: <https://pymongo.readthedocs.io/en/stable/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 11 de junio de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.